topics, and multiple producers send messages on the same topic.

- The consumers register with the broker for messages on one or more topics.
- The producers send the messages only to the broker, and not to the consumers. The broker, in turn, delivers the messages to all the consumers that are registered against the topic.
- The producers do not expect any response from the consumers. In other words, the producers and consumers do not block each other.

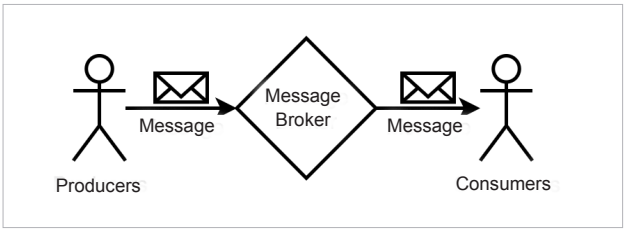

Figure 1: Asynchronous messaging

There are several message brokers available in the market, and Apache Kafka is one of the most popular among them.

Apache Kafka

Apache Kafka is an open source distributed messaging system with streaming capabilities, developed by the Apache Software Foundation. Architecturally, it is a cluster of several brokers that are coordinated by the Apache Zookeeper service. These brokers share the load on the cluster while receiving, persisting, and delivering the messages.

Partitions: Kafka writes messages into buckets known as partitions. A given partition holds messages only on one topic. For example, Kafka writes messages on the topic heartbeats into the partition named *heartbeats-0*, irrespective of the producer of the messages.

However, in order to leverage the cluster-wide parallel processing capabilities of Kafka, administrators often create more than one partition for a given topic. For instance, if the administrator creates three partitions for the topic *heartbeats*, Kafka names them as *heartbeats-0*, *heartbeats-1*, and *heartbeats-2*. Kafka writes the heartbeat messages across all the three partitions in such a way that the load is evenly distributed.

There is yet another possible scenario in which the producers associate each of the messages with a key. For example, a component uses C1 as the key while another component uses C2 as the key for the messages that they produce on the topic heartbeats. In this scenario, Kafka makes sure that the messages on a topic with a specific key are always found only in one partition. However, it is quite possible that a given partition may hold messages

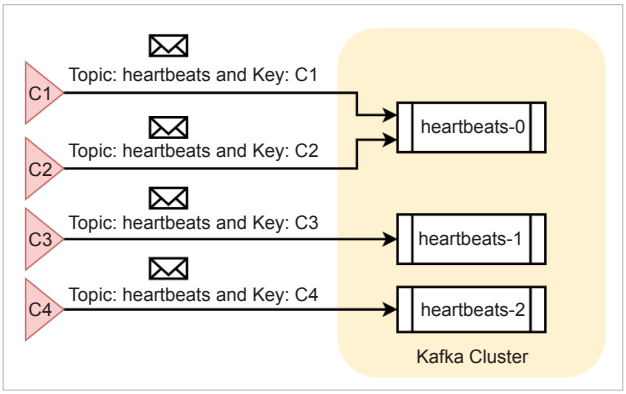

Figure 2: Message distribution among the partitions

with different keys. Figure 2 presents a possible message distribution among the partitions.

Leaders and ISRs: Kafka maintains several partitions across the cluster. The broker on which a partition is maintained is called the leader for the specific partition. Only the leader receives and serves the messages from its partitions.

But what happens to a partition if its leader crashes? To ensure business continuity, every leader replicates its partitions on other brokers. The latter act as the in-sync-replicas (ISRs) for the partition. In case the leader of a partition crashes, Zookeeper conducts an election and names an ISR as the new leader. Thereafter, the new leader takes the responsibility of writing and serving the messages for that partition. Administrators can choose how many ISRs are to be maintained for a topic.

Message persistence: The brokers map each of the partitions to a specific file on the disk, for persistence. By default, they keep the messages for a week on the disk! The messages and their order cannot be altered once they are written to a partition. Administrators can configure policies like message retention, compaction, etc.

Consuming the messages: Unlike most other messaging systems, Apache Kafka does not actively deliver the messages to its consumers. Instead, it is the responsibility of the consumers to listen to the topics and read the messages. A consumer can read messages from more than one partition of a topic. And it is also possible that multiple consumers read messages from a given partition. Kafka guarantees that no message is read more than once by a given consumer.

Kafka also expects that every consumer is identified with a group ID. Consumers with the same group ID form a group. Typically, in order to read messages from N number of topic partitions, an administrator creates a group with N number of consumers. This way, each consumer of the group reads messages from its designated partition. If the group consists of more consumers than the available partitions, the excess consumers remain idle.